

TP 4
Sockets

Programación sobre redes

Instituto Ind. Luis A. Huergo

Ignacio Vigilante

5 de octubre de 2026

Índice

1 Introducción **2**

1.1 El Gran Teatro Mecánico 2
1.2 El teatro 2
1.3 El instrumento y el músico 3
1.4 Objetivo 3

2 Consigna **4**

2.1 Lo que provee la cátedra 4
2.2 Lo que tienen que hacer 4
2.3 Decisiones de diseño 5
2.4 Requisitos mínimos 6
2.5 Adicionales 6
2.6 Informe 6
2.7 Librerías externas 6

3 Rúbricas **8**

4 Entrega **9**

4.1 Grupos 9
4.2 Qué se entrega 9
4.3 Fechas 9
4.4 Lenguajes y soporte 10
4.5 Uso de inteligencia artificial 10

1 Introducción

1.1 El Gran Teatro Mecánico

En la calle Corrientes hay un teatro, el **Gran Teatro Mecánico**, que en lugar de músicos tiene una orquesta de **autómatas**: muñecos de bronce y madera que leen la partitura, siguen al director y tocan instrumentos de verdad. Los fabricaba un relojero que ya no está, y el teatro necesita músicos nuevos para la próxima temporada. Los va a fabricar la escuela: **un autómata por grupo**.

Cada autómata tiene un **cerebro**, que lee la partitura y sigue al director, y un cuerpo. En los vientos, las dos manos digitan y los **pulmones** soplan; en las cuerdas, la mano izquierda digita y la derecha mueve el **arco** o la púa. Cada parte del cuerpo tiene su propio mecanismo, y las partes se comunican entre sí por cable. El **instrumento** es otro aparato, que recibe los movimientos de los dedos y el aire y produce el sonido. El **director** está en el foso del teatro, y también se comunica con los músicos por cable.

Los cables del teatro son viejos: a veces pierden lo que llevan, a veces lo entregan dos veces y a veces lo entregan desordenado. El autómata tiene que tocar bien igual.

1.2 El teatro

El **teatro** lo provee la cátedra. Adentro está el director, que reparte a cada músico su parte, lleva el reloj, da las señas (tempo, dinámica, parar, retomar desde un compás), recibe lo que toca cada uno y lo hace sonar en una página web, la sala. También mide cuántas notas llegaron a tiempo, cuántas sonaron sucias y cuánto tardó cada músico en responder a una seña.

1.3 El instrumento y el músico

El **instrumento** también lo provee la cátedra, y corre en la máquina de cada grupo. Es un programa que recibe por red los movimientos de los dedos y la energía (el aire o el arco), simula algunas reglas físicas de un instrumento real (por ejemplo, mover un dedo lleva tiempo, y soplar mientras un dedo se está moviendo suena mal) y le avisa al teatro qué está sonando. El **músico** es lo que tienen que programar ustedes.

1.4 Objetivo

El objetivo de este trabajo es aplicar lo que vimos en las últimas clases: **sockets**, usando **TCP** y **UDP** donde corresponda cada uno, y **multiplexación** de entrada y salida. El cerebro tiene que atender al director, al reloj y a dos o tres partes del cuerpo a la vez, así que no puede quedarse bloqueado esperando a una sola de ellas.

Este trabajo tiene menos consigna que los anteriores. Lo que está fijo es el **contrato** con el teatro y el instrumento, que está en un documento aparte, y unas pocas reglas sobre el músico (sección 2.2): el cerebro y cada parte del cuerpo son procesos separados y se comunican por sockets. El resto queda a su criterio: cómo se reparten el trabajo los procesos, qué sockets usan entre ellos, qué protocolo definen, qué lenguaje y qué modelo de concurrencia usan. La nota depende sobre todo de esas decisiones y de cómo las justifican.

2 Consigna

2.1 Lo que provee la cátedra

En el repositorio del grupo van a encontrar:

- `contrato.md`: el **contrato**. Describe todo lo que el músico necesita para comunicarse con el teatro y con el instrumento: los mensajes, las reglas físicas, las tablas y qué se mide. Conviene leerlo completo antes de empezar.
- `teatro/`: el teatro, para correr en su máquina mientras desarrollan. Sirve la sala, una página donde se ve y se escucha lo que tocan.
- `instrumento/`: el instrumento, con opciones para simular pérdida, duplicados y desorden.
- `tablas/`: las tablas de digitación de los ocho instrumentos, con las formas de acorde de la guitarra, y la del piano, que es opcional.
- `obras/`: una escala para probar, un muestrario con cada instrumento solo y varias obras.
- `probar.py`: el **verificador**. Corre su músico contra el teatro y el instrumento, con y sin caos, y genera el informe que usamos para corregir.
- `cheatsheet.md`, `cheatsheet_python.md` y `cheatsheet_js.md`: UDP y multiplexación en C++, Python y JavaScript, que no llegamos a practicar en el taller. En `ejemplo/` hay un ping-pong UDP en los tres lenguajes.

No hay código del músico: lo programan ustedes desde cero.

2.2 Lo que tienen que hacer

Tienen que programar el **músico**: un **cerebro** y las partes del cuerpo que corresponden a su instrumento según la tabla. Las reglas son las siguientes:

1. El cerebro y cada parte del cuerpo son **procesos separados**: en los vientos, las dos manos y los pulmones; en las cuerdas, la mano izquierda y el arco o la púa.

2. Los procesos se comunican entre sí **por sockets**. El dominio y el tipo de socket los eligen ustedes. No se pueden usar pipes, archivos ni memoria compartida.
3. Cada parte del cuerpo le habla al instrumento **desde su propio socket**. El instrumento lo controla y se lo informa al teatro.
4. El músico se levanta con un único script, `correr.sh`, con la interfaz que define el contrato. El verificador solo conoce ese script.
5. El músico tiene que poder tocar **cualquiera de los ocho instrumentos** a partir de su tabla.
6. Pueden usar **el lenguaje que quieran**, y el lenguaje es una de las decisiones a justificar (sección 2.3). Tiene que correr en Linux siguiendo el `README` del grupo.

2.3 Decisiones de diseño

La mayor parte de la nota sale de estas decisiones. Cada una tiene más de una respuesta razonable; lo que se evalúa es qué eligieron y cómo lo justifican.

- **Lenguaje y modelo de concurrencia.** Threads, sockets no bloqueantes o multiplexación, y por qué. El lenguaje condiciona bastante esta decisión: en Go lo habitual son las goroutines, Node trabaja con un event loop, en Python se puede elegir y en C++ hay que escribir el `poll` a mano. Tienen que mostrar con `strace` **en qué syscall espera cada uno de sus procesos** y explicar a qué modelo corresponde.
- **Transporte interno.** `AF_UNIX` o `AF_INET`, stream o datagramas, y cómo delimitan los mensajes. Qué pasaría si una parte corriera en otra máquina.
- **Quién agenda.** Si el cerebro decide cuándo se mueve cada dedo y las partes solo ejecutan, o si cada parte recibe su parte de la partitura y el beat y se organiza sola. Cómo llega el reloj del director hasta cada parte.
- **Anticipación.** Un dedo tarda en llegar a su lugar: cuánto antes del beat hay que moverlo, y cómo se aseguran de cortar la energía antes de mover los dedos si eso lo hacen procesos distintos.
- **Eco.** Qué hacen con lo que les devuelve el instrumento: cuándo reenvían, con qué `seq`, y cómo distinguen una acción que se perdió de una que llegó pero quedó mal. Reenviar de más genera torpezas, y reenviar de menos, notas sucias.
- **Notas simultáneas.** En las cuerdas hay acordes y bajos que siguen sonando mientras empieza otra nota. La digitación se busca en la tabla, pero la mano tiene que cubrir todo lo que suena a la vez, y hay que soltar una cuerda antes de mover el dedo que la está pisando.
- **Fallas.** Qué pasa si se cae una mano en el medio de la obra, si el instrumento arranca después que el músico o si el teatro se cae y vuelve.
- **Señas.** Cómo llega un `PARAR` o un `DINAMICA` desde el cerebro hasta la parte que tiene que actuar, y cuánto tarda.

2.4 Requisitos mínimos

Para aprobar, el músico tiene que:

- tocar la escala y la obra con el teatro y el instrumento locales, **sin caos**, con cualquiera de las ocho tablas: todas las notas a tiempo, sin sucios ni torpezas;
- seguir tocando con **10% de pérdida** en el instrumento: la gran mayoría de las notas a tiempo y ninguna faltante;
- tocar la **guitarra con acordes** (el verificador la prueba con una obra de bajo y acordes);
- responder a las seis señas del director **en menos de un beat**.

Los umbrales exactos están al principio de `probar.py`. El nivel más difícil del verificador, con 20% de pérdida, desorden y duplicados, no es necesario para aprobar, pero suma para la nota (capítulo 3). Si un nivel no pasa, el verificador lo repite hasta tres veces y se queda con el mejor intento; nosotros corregimos de la misma forma.

2.5 Adicionales

Si quieren ir más allá, en orden aproximado de dificultad:

- Que el músico se recupere si se mata una parte del cuerpo en el medio de la obra. Pueden probarlo con `kill`.
- Correr las partes del cuerpo en máquinas distintas, por ejemplo una por integrante del grupo.
- Tocar el **piano**: una voz por dedo, y sin una tabla que diga qué mano y qué dedo van en cada tecla.

2.6 Informe

Junto con el código entregan un informe corto en PDF con:

- un diagrama de los procesos del músico y los sockets que los conectan, con el dominio y el tipo de cada uno;
- cada decisión de la sección 2.3: qué eligieron, qué descartaron y por qué;
- la salida de `strace` que muestra en qué syscall espera cada proceso;
- los resultados del verificador en cada nivel;
- los problemas que encontraron y cómo los resolvieron, incluidos los que no pudieron resolver.

2.7 Librerías externas

El uso de librerías externas está permitido bajo ciertos criterios:

- Las librerías no pueden resolver una porción significativa de lo que se está evaluando, sí partes superfluas del trabajo. Una librería de sockets de más alto nivel que la del lenguaje cuenta como resolver lo que se evalúa.
- Se debe justificar el uso de cada librería y su impacto en la solución propuesta.
- No deben requerir instalaciones externas o configuraciones especiales para compilar y ejecutar el proyecto más allá de lo que diga el `README`.

Ante cualquier duda, pregunten antes de usarlas.

3

Rúbricas y criterios de corrección

Los criterios de corrección, de mayor a menor peso:

- **Decisiones y criterios de diseño**: es el criterio principal. Se evalúa que cada decisión de la sección 2.3 esté tomada a conciencia y justificada en el informe. Una decisión bien justificada vale más que una solución perfecta sin justificar, y también se valora haber cambiado una decisión después de medir que la primera no funcionaba.
- **Funcionamiento**: lo que mide el verificador en sus cuatro niveles (sin caos y con señas, 10% de pérdida, 20% de pérdida con desorden y duplicados, y guitarra con acordes): notas a tiempo, sucios, torpezas y tiempo de respuesta a las señas.
- **Adecuación al contrato y a las reglas**: se cumplen el contrato y las reglas de la sección 2.2, y `correr.sh` levanta todo en Linux siguiendo el `README` del grupo. Después de un `FIN` no tiene que quedar ningún proceso vivo.
- **Prolijidad, legibilidad y buenas prácticas**, en menor medida: cerrar lo que no se usa, no bloquear donde no corresponde y que el protocolo interno se entienda leyendo el código.

4 Información sobre la entrega

4.1 Grupos

El trabajo se hace en grupos de 2 o 3 integrantes, **sin excepciones**. Recomendamos **no dividirse** las tareas: el trabajo está pensado para hacerse con *pair programming*, y en este caso el cerebro y las partes del cuerpo tienen que compartir un protocolo que conviene definir entre todos.

4.2 Qué se entrega

La entrega es el repositorio del grupo en Classroom 50, que tiene que tener:

- el código del músico y `correr.sh`;
- un `README` con lo que hay que instalar y cómo se corre en Linux;
- el informe en PDF (sección 2.6);
- un tag `entrega` en el commit que quieren que corrijamos.

Hagan **commits a medida que avanzan** y no uno solo al final: les sirve para volver atrás cuando algo deja de andar, y a nosotros para la corrección. Los commits posteriores a la fecha límite hacen que el trabajo se considere entregado **tarde**.

4.3 Fechas

- **Presentación de la consigna**: lunes 5 de octubre. Ese día queda publicado todo el material (contrato, teatro, instrumento, tablas, obras y verificador), y no se agrega nada después.
- **Entrega**: sábado 31 de octubre, a las 23:59.

Para que no les quede todo para el final, les sugerimos estos hitos. No se entregan, pero conviene llegar a cada uno en la fecha indicada:

1. **Lunes 12 de octubre**: los procesos del músico se levantan con `correr.sh` y se conectan entre sí; el cerebro se registra en el teatro, recibe su parte y sigue el reloj.
2. **Lunes 19 de octubre**: las manos y la energía hablan con el instrumento y usan el eco. Suena la escala en un viento y en una cuerda, y pasa el nivel 0 del verificador.
3. **Lunes 26 de octubre**: reenvíos con caos, las señas del director y los acordes de la guitarra. Pasan los niveles 1 y 3.

4.4 Lenguajes y soporte

Pueden usar cualquier lenguaje. La cátedra da **soporte completo** para C++, Python y JavaScript o TypeScript. Con otros lenguajes los ayudamos en lo que podamos, pero si algo no funciona y no lo podemos reproducir, lo tienen que resolver ustedes. En todos los casos tiene que correr en Linux siguiendo el `README`.

4.5 Uso de inteligencia artificial

Pueden usar modelos de lenguaje para hacer **preguntas**: qué hace una syscall, por qué falla algo, cómo se usa una función. Lo que **no pueden** es usarlos para que hagan el trabajo, ni el código ni el informe. Se nota en el resultado, en el historial de commits y cuando les pedimos que expliquen sus decisiones. Un trabajo hecho por un modelo se considera no entregado.